# Performance-Aware Leader Selection in Geo-Distributed BFT Consensus

Lucca Cézar[1][0000−1111−2222−3333], Fernando Dotti[1][1111−2222−3333−4444], and Fernando Pedone[2][2222−−3333−4444−5555]

[1] PUCRS - School of Technology - PPGCC, Porto Alegre, Brazil
lucca.dornelles99@edu.pucrs.br, fernando.dotti@pucrs.br
[2] Università della Svitzzera italiana, Lugano, Switzerland
fernando.pedone@usi.ch

**Abstract.** Blockchain consensus protocols have increasingly focused on performance improvements, including approaches such as better leader selection. However, the specifics of how a leader impacts consensus latency remains poorly characterized. We conduct an experimental study with geo-distributed topologies and show that the choice of leader can significantly impact consensus, affecting step durations, quorum formation, and increasing consensus time by up to 40%. Based on these findings we propose a simple, generic mechanism that uses distributed observations to steer leader selection toward higher-performing leaders. Our approach is robust to Byzantine behavior, incurs no worst-case performance overhead, and achieves throughput gains of up to 80% compared to node-removal baselines, without reducing Byzantine tolerance. While evaluated on a specific BFT protocol, the method generalizes to rotating-leader SMR systems.

**Keywords:** Byzantine consensus · Blockchains · Leader selection.

## 1 Introduction

Early blockchain platforms such as Bitcoin [9] and Ethereum [12] demonstrated fully functional permissionless distributed consensus systems, prioritizing security and correctness at the expense of performance. As modern systems experience increasing workloads, this creates a need for higher throughput and lower per-transaction latency. In response, modern research has largely shifted towards improving performance, sometimes even assuming lower security thresholds for the sake of performance [6].

Consensus algorithms are a central piece in blockchains. Several modern such algorithms employ rotating leaders or proposers, aiming to mitigate the performance impact of byzantine processes; avoid censorship; and provide fair reward distribution. While a leader coordinates the different consensus phases of a consensus instance (e.g. as in [15]), a proposer is only in charge of proposing a value for an instance and the further phases are distributed (e.g. as in [1]). In this paper we explore proposer selection as a possible avenue for performance improvement.

Although not addressed in this paper, it is expected that leader-based algorithms would be even more affected by the topology than proposer-based ones.

*A first contribution* of this paper is the set of experiments to understand the impact of the proposer's position within the topology on the performance of geo-distributed consensus. Our experiments in Section 4 show that the position of a proposer is a significant factor throughout the entirety of consensus. In some topologies we observe overheads of up to 40% in overall consensus latency, attributed only to the position of a proposer. We also show that the position of a proposer impacts quorum formation, and therefore the duration of individual consensus steps.

*The second contribution* of this paper is, based on the above observations, the proposition of a generic proposer selection optimization mechanism. In Section 5 we define this mechanism, argue for its applicability in Byzantine environments, and evaluate it through practical experiments. Our experiments in Section 6 compare a Tendermint implementation with and without our mechanism, showing throughput improvements of up to 15% in some topologies with no impact to Byzantine tolerance.

The paper follows in Section 2 with the system model, a discussion of Tendermint and related work. It then continues with Sections 4, 5 and 6 as discussed above, followed then by Conclusions.

## 2 Background

### 2.1 System Model

We consider a distributed system composed of an unbounded set of client processes that submit transactions (or operations) to a bounded set $\Pi$ of $N$ server processes. For simplicity, here we consider that all nodes in $\Pi$ act as *validators* (i.e., nodes that execute the consensus protocol). The set of nodes is dynamic and known by all nodes.

Nodes communicate by message passing. The system is partially synchronous: it is initially asynchronous when there is no bound on message delays or relative process speeds. At an unknown moment (i.e., the Global Stabilization Time), the system becomes synchronous, when message delays and processing speed become bounded.

There are up to $f < N/3$ *faulty* or Byzantine nodes; the remaining nodes are *honest*. Faulty nodes can deviate from their specifications, whereas honest nodes adhere to them. Cryptographic techniques are used, with adversaries not having (with high probability) the computational power to subvert them.

### 2.2 Blockchain Consensus

We consider permissioned consensus protocols, such as PBFT [2] and Tendermint [1]. Tendermint consensus operates through three sequential phases, *propose*, *prevote*, and *precommit*, which functionally correspond to the *pre-prepare*,

*prepare*, and *commit* stages of the Practical Byzantine Fault Tolerance (PBFT) algorithm. A primary distinction of Tendermint is the implementation of a rotating proposer mechanism: by using a round-robin approach in which each node proposes only once, the network mitigates censorship [13].

Furthermore, unlike PBFT, Tendermint is designed for environments where unknown participants can join. To protect against Sybil attacks [4], where a malicious actor might otherwise flood the network with nodes to gain control, the protocol integrates a Proof of Stake (PoS) mechanism. Participants receive rewards for consensus participation, while detected malicious behavior results in the destruction or reduction of their stake via slashing. This design effectively increases the cost of subversion, making it expensive for an adversary to acquire a quorum.

## 3 Related Work

Several works have explored performance-aware leader selection in Byzantine fault-tolerant consensus. Existing proposals differ substantially in both their objectives and integration strategies. To better position our contribution, we classify prior work along seven dimensions (see Table 1): **(I) Score Representation**, distinguishing between continuous scores and discrete node classes; **(II) Committee Eviction,** indicating whether nodes may be removed from the active committee; **(III) Evaluation Metrics**, describing the information used to estimate node quality; **(IV) Communication Overhead**, indicating whether the mechanism requires additional protocol messages or coordination phases; **(V) Decoupled Architecture**, capturing whether the optimization is implemented independently from the consensus core and can therefore be reused across different SMR protocols; **(VI) Continuous Evaluation**, indicating whether node quality is continuously updated during execution; and **(VII) Rotation Policy**, describing whether proposer rotation remains active after optimization.

Most related proposals are derived from PBFT-style primary-backup architectures and therefore focus on improving view-change behavior after detecting a slow or faulty leader (e.g., [6, 11, 17, 19]). In these systems, node differentiation primarily serves as a ranking mechanism for selecting replacement leaders after a disruption. Proposer optimization is often reactive rather than adaptive.

A second recurring characteristic is the reliance on committee eviction to sustain performance (e.g., [5, 7, 10, 14, 18]). These approaches remove nodes considered slow, unstable, or suspicious from the active consensus set. While effective at increasing throughput in controlled environments, committee reduction introduces a direct trade-off between performance and Byzantine resilience. Removing slow but correct nodes reduces the effective committee size, thereby lowering the maximum number of Byzantine faults the system can tolerate. Moreover, modern blockchain deployments already incorporate mechanisms to detect and exclude malicious validators, making additional performance-oriented eviction layers potentially redundant and operationally risky.

| Algorithm | Score Type | Committee Eviction | Evaluation Metrics | Communic. Overhead | Decoupled Architecture | Continuous Evaluation | Rotation Policy |
|---|---|---|---|---|---|---|---|
| [5] | C | Y | Consensus participation and proposal success ratio | Y | Y | Y | N |
| [6] | C | N | Per-node consensus latency measured through active probing by clients | Y[3] | Y | N | N |
| [7] | D | Y | Composite reputation derived from performance and participation | Y | N | Y | N |
| [10] | D | Y | Latency, availability, activity level, and consensus contribution | Y | N | Y | N |
| [11] | D | N | Behavioral classification and response-time analysis | Y | N | Y | N |
| [14] | C | Y | Latency measurements combined with reputation signals | Y | N | Y | N[4] |
| [16] | C | N | Observed leader throughput during evaluation windows | N | Y | N | N |
| [17] | C | N | Historical leader success, measured by committed blocks | Y | Y[5] | Y | N |
| [18] | D | Y | Binary success/failure outcomes of consensus participation | N | N | Y | N |
| [19] | C | N | Consensus participation consistency, and failure frequency | Y | N | Y | N |
| This paper | C | N | Consensus duration | Y | Y | Y | P |

Table 1: Comparison of leader selection strategies

Legend: Continuous (C) scores allow fine-grained proposer differentiation, while Discrete (D) scores classify nodes into categories. Committee Eviction indicates whether nodes may be removed from the active consensus committee. Proportional (P) rotation biases proposer frequency according to performance, whereas No rotation (N) maintains the primary/proposer until a view change is proposed.

The surveyed literature also differs in how it treats proposer fairness. Many PBFT-derived optimizations effectively converge toward fixed or weakly rotating leaders, increasing susceptibility to censorship and centralization. Our approach instead adopts proportional prioritization: higher-performing nodes are selected more frequently, while all nodes remain eligible proposers. This preserves continuous rotation while still exploiting topology-aware performance asymmetries. [OUR APPROACH CAN, DEPENDING ON THE CONFIGURED VALUES (E.G. ALLOWING SCORE TO BE UP TO +-100% OF STAKE), REMOVE NODES FROM ROTATION (BUT KEEP THEM IN THE COMMITTEE). THIS IS INTENTIONAL AND ALLOWS FOR MAXIMUM THROUGHPUT AT THE COST OF CENSORSHIP RESILIENCE]

[3] Multiple consensus.

[4] The [14] primary is the node with the most voting power. If voting power changes, a new node can become the primary even without suspected failure. However, the algorithm does not perform intentional leader rotation

[5] For systems without leader rotation.

Finally, existing work generally treats proposer performance as an abstract property associated with computational speed or node responsiveness, with limited investigation into how network topology shapes consensus execution. Our experiments show that proposer placement directly affects overall consensus latency, influencing the duration of subsequent protocol phases. In geo-distributed deployments, proposer location therefore becomes a first-order determinant of consensus efficiency rather than merely a secondary implementation detail.

Table 1 highlights two dominant trends in prior work: (i) performance optimization through committee reduction, and (ii) tightly coupled protocol-specific leader management. Our approach differs in that it preserves the full committee and the original Byzantine threshold while continuously adapting proposer frequency based on observed consensus performance. Unlike eviction-based strategies, proposer prioritization improves throughput without sacrificing resilience or requiring invasive protocol modifications.

## 4 Understanding the Impact of Proposer Location

This section investigates how proposer's network latency relative to other nodes affects overall consensus performance. We discuss how to measure consensus latency under different proposers; report experiments in a geo-distributed setting; show evidence that proposer differentiation is possible; and discuss the influence of different proposers on quorum formation.

We conducted experiments on the CloudLab[6] testbed using 13 m510[7] nodes. Each node emulated a distinct AWS zone, with inter-node latencies[8] injected via the `tc`[9] (traffic control) utility to represent a realistic geo-distributed topology. The study utilized the Malachite implementation of the Tendermint protocol. Each execution lasted five minutes, including a 30-second warm-up and 15-second cool-down, with clock synchronization maintained via NTP to ensure deviations remained below 10ms.

Our primary observation metric for evaluating proposer location is the total time elapsed from the moment the proposer sends the proposal to the final decision at each node. As this metric relates timestamps at different nodes, NTP is used. We call this metric *global consensus duration*.

As we also want to propose a mechanism for distributed monitoring each proposer's performance, we would like to avoid measures involving the proposer and each node. Such measures both depend on synchronized nodes and also are subject to dishonest proposers lying about their timestamps to forge fast latency samples in their favor. Thus we introduce a second metric, which is measured locally at each node. When a node finishes height $h-1$ it locally enters the proposal phase of height $h$. This is the moment the latency sample initiates.

[6] `https://cloudlab.us/`

[7] `https://docs.cloudlab.us/hardware.html#(part.cloudlab-utah)`

[8] `https://github.com/cometbft/cometbft/blob/v2.x/test/e2e/pkg/files/aws-latencies.csv`

[9] https://www.man7.org/linux/man-pages/man8/tc.8.html

A proposer should then send the proposal while a non-proposer awaits for the proposal. Afterwards, when decision is reached, the sample finishes. We call this metric the *local consensus duration*.

**Topology with uniform latencies among node pairs.** Our experimental methodology followed a sequence of targeted tests. First, a *sanity check* was performed using a 4-node topology with a uniform 100ms latency between any pari of nodes. We measured the global consensus duration for each possible proposer and node pair. The results showed negligible variation ($< 1.6\%$) across samples, indicating that any subsequent performance discrepancies were strictly due to network topology. Also, we observed no impact in switching proposers at each decision or after a number of decisions. For better legibility, we show figures with results switching proposers every 10 decisions.

**Proposer Impact on Consensus Duration.** Figure 1.a depicts total consensus latency in our 13-node geo-distributed topology. It shows that the latency perceived at each node clearly is impacted by the proposer (one proposer per column with different color). [??? HERE INSERT NUMERICAL DISCUSSION WITH DATA FROM THE PICTURE. THIS DISCUSSION SHOLD SUPPORT THAT PROPOSER DIFFERENTIATION IS POSSIBLE ] Moreover, it also indicates that proposer differentiation should be possible since, for some proposers, latencies observed are ...

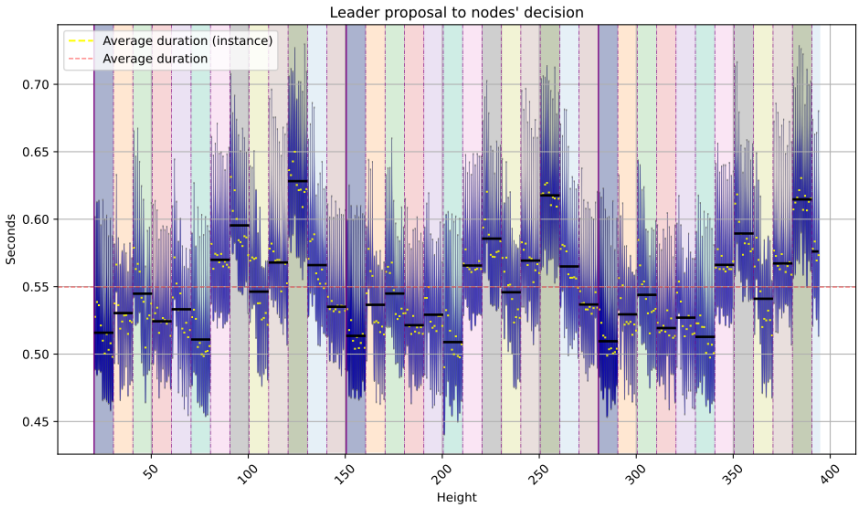
As previously discussed, the local consensus duration is a measure of practical interest. Figure 1.b depicts this measure for the same data. As can be observed, the local consensus duration measure is strongly related to the global consensus duration... [??? HERE INSERT NUMERICAL DISCUSSION WITH DATA FROM THE PICTURE. THIS DISCUSSION SHOULD SUPPORT THAT THE LOCAL MEASURE IS STRONGLY RELATED TO THE GLOBAL AND THAT IT ALLOWS PROPOSER DIFFERENTIATION. ]

[ cdfs da figura 1.a ]

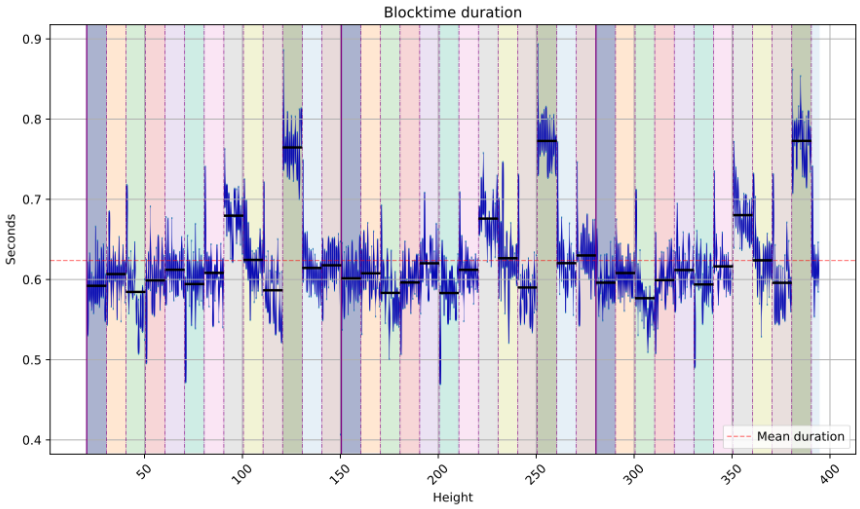
[ titulo p 4 ]

[ tail latency nos resultados??? nao esta aparecendo ... ]

[ VER SE ISSO FICA ] To evaluate the statistical significance of our data, we performed a Kruskal–Wallis test [8] on both the consensus duration and individual step durations. [INDIVIDUAL STEP COTINUA NA DISCUSSAO?] [ACHO IMPORTANTE PELO MENOS MENCIONAR QUE O PROPOSER AFETA OS STEPS INDIVIDUAIS, JÁ QUE ISSO IMPACTA TODO O FUNCIONAMENTO] Kruskal-Wallis is a non-parametric statistical test used to determine whether samples across groups originate from the same distribution. We use this test to determine whether our groups (different proposers) produce statistically significant differences in the observed data (durations). The test produces a p-value, representing the probability of observing the results under the null hypothesis (i.e. that the different proposers do not affect the data), and an H-statistic, which measures how strongly the observed distributions deviate from the null hypothesis. We also calculated the effect size using $\epsilon^2$ (epsilon squared),




(a) Each vertical line represents a consensus instance and is composed by global consensus duration samples, from the respective proposer to each node. Line darkness represents concentration of samples.




(b) Each vertical line represents a consensus instance and is composed by local consensus duration samples at each node, under the respective proposer.

Fig. 1: (a) Global consensus duration. (b) Local consensus duration. 10 instances per proposer, each proposer has a different color, divided by vertical dotted lines. Solid line indicates a new round through same proposers - repeating the pattern.

representing the relative magnitude of the differences between groups, ranging between 0 and 1, with larger values indicating stronger group effects.

For each metric, we evaluated both quorum metrics (first $2F + 1$ nodes terminate consensus or finish a step), and the local view of each individual node for those same metrics.

We grouped executions by proposer (k = 13), with a total of 344 consensus instances (n = 344). All tests yielded statistically significant results ($p < 0.001$). For tests on the fastest quorum, we observed $H > 200$ with $\epsilon^2 > 0.5$ for the step durations, and $H = 153$ with $\epsilon^2 = 0.43$ for the overall consensus duration. These results indicate substantial variability in both consensus and step durations associated with changes in proposer.

When analyzing individual nodes, consensus duration still showed significant association with proposer selection, with H-values ranging from 183.38 to 218.90 and corresponding $\epsilon^2$ values between 0.52 and 0.62. For the prevote step, the H-statistic varied from 97.13 to 307.46, with one outlier at H = 53.74. The corresponding $\epsilon^2$ values ranged from 0.13 to 0.89, with $\epsilon^2 = 0.26$ for the case where H = 97.13. For the precommit step, H-values ranged from 110.71 to 298.32, with $\epsilon^2$ values between 0.30 and 0.86.

These results demonstrate that the proposer has a considerable effect on both consensus duration and step durations across all nodes in the network.

**Proposer Impact on Quorum Formation.** After the proposal, further communication steps are all-to-all. While the time to receive a quorum, in all-to-all communication phases, theoretically depends on the location of each receiver node, we investigated whether the proposer's location could influence such steps. To check this, we computed and evaluated the quorums built at each node, rotating the whole set of proposers several rounds. We observed a 98% consistency in each node's local quorum composition when the same node acted as proposer. Nodes geographically closer to the proposer receive the proposal faster and, consequently, their votes are the first to arrive and form the *prevote* quorum.

[MOSTRAR A FIG 'IMG/NODE4 QUORUMS.PNG' ?] As stated in the Tendermint specification, if a farther node manages to join another node's prevote quorum, that second node will wait for that node in the *precommit* step even if a faster quorum is reached (this is intended for Proof of Stake rewards). This means that in the precommit phase, a specific node cannot progress with it's ideal quorum (set of closest nodes), negatively impacting latency and throughput. This also leads to the high consistency among prevote and precommit quorums in a specific node.

# 5 Performance-Aware Proposer Prioritization

We propose a generic mechanism that continuously adapts the proposer frequency based on the observed performance of consensus instances. The mechanism targets consensus protocols that employ rotating proposers and maintain a deterministic proposer-selection function, denoted by *nextProposer* (e.g., derived from stake in proof-of-stake systems). The approach operates in three stages: nodes first locally classify proposers according to observed consensus latency (Section 5.1); these observations are then reconciled into a globally consistent classification history (Section 5.2); finally, the resulting classifications are incorporated into proposer selection, biasing proposer frequency toward nodes that consistently lead to faster consensus decisions while preserving continuous rotation and Byzantine resilience (Section 5.3). We conclude the section by discussing the properties and correctness of the proposed mechanism (Section 5.4).

## 5.1 Sampling and classifying proposers

Each node continuously monitors the latency of consensus instances in order to estimate the relative performance of proposers. For each consensus height, each node locally measures the elapsed time from the start of the consensus instance (e.g., Tendermint's `propose`) to the final decision.

Since each consensus instance is associated with a known proposer, the observed latency provides an estimate of that proposer's effectiveness at driving consensus progress.

Each node maintains a sliding window $L$ containing the last $k$ observed consensus latencies, where $k$ is a configuration parameter. From this window, the node continuously computes the average consensus latency $\bar{L}$. A proposer is then classified relative to this average latency.

Given a latency sample $s$, the proposer is classified as:

$$fast \quad \text{if } s \leq \bar{L}$$

$$slow \quad \text{if } s > \bar{L}$$

We denote by $c_h \in \{fast, slow\}$ the resulting classification for a consensus height $h$.

## 5.2 From local to global classifications

We now show how to transform local classifications into a globally consistent classification that can later be used to bias proposer selection. Rather than introducing a separate consensus protocol, our approach piggybacks classification information onto the existing consensus execution. The mechanism operates continuously through two lightweight steps: *vote*, *propose*, and *adopt*.

*Vote step.* Once the node reaches a decision on height $h$ that nodes calculates the consensus duration and compares it to the average to obtain its local $c_h$ vote for that height. This value is only calculated once and a honest node will never change a previous vote for any reason. The node then sends its local $c_h$ along with the first consensus message (e.g. tendermint's `prevote`) for the height $h+1$, thereby avoiding additional communication rounds.

This vote contains the height which it refers to, its vote, and a criptographic signature for the vote. [MENTION THAT HAVING A SIGNATURE JUST FOR THE VOTE SIMPLIFIES DECISION PROPOSAL? E.G. PROPOSER CAN JUST SEND VOTE,HEIGHT AND SIGNATURES FOR THAT PAIR TO PROVE IT]

*Choose step.* At the beginning of height $h + 2$, the proposer evaluates the classifications received for the proposer of height $h$, reaching one of three states

1. The proposer receives enough votes for a decision $d_h \in \{fast, slow\}$; It then proses $d_h$.
2. The proposer receives at least $F + 1$ votes for $fast$ and at least $F + 1$ votes for slow, making a decision impossible; It then proposes $\perp$ (non-decision).
3. The proposer does not receive enough votes to reach a decision; Consensus proceeds as normal but without any decision attached.

In cases 1 and 2 the proposer must send valid signatures to validate its proposal. The lack of signatures, or invalid signatures, is a Byzantine behavior which invalidates the block and must be detected by existing Byzantine detection mechanisms.

Should this decision fail (case 3), any future proposer can then re-propose a past $d_h$ value for any height $h$ with no decision. [MENTION THAT IN PRACTICE THIS SHOULD BE LIMITED TO CURHEIGHT-N?] If nodes reach different decisions (e.g. due to Byzantine equivocation), the value that is proposed and accepted first is accepted.

*Adopt step.* After a consensus with a valid $d_h \in \{fast, slow\}$ is committed, each node updates its local score accordingly, changing its internal values for the proposer selection algorithm. As a consequence, all honest nodes eventually converge to the same proposer classification history.

[DO WE STILL NEED TO EXPLAIN THIS NOW THAT WE ARE GOING WITH A BINARY DECISION?] To understand the need for $2f + 1$ matching classifications, let $k$ be the number of votes the proposer receives, where $k = k_{fast} + k_{slow}$At most $f$ of these votes are from Byzantine processes, and there are votes from $(N-k)$ nodes that the proposer does not know about. Therefore, to guarantee that there are votes from a majority of honest nodes for *fast* (resp., *slow*), it must be that $k_{fast} - f > k_{slow} + k_{\perp} + (N - k_{fast} - k_{slow})$, or $k_{fast} \geq \lceil (N + f)/2 \rceil$, and since $N = 3f + 1$, $k_{fast} \geq 2f + 1$.

### 5.3 Continuous proposer scoring and prioritization

The goal of the scoring mechanism is to slightly bias proposer selection toward nodes that consistently lead to faster consensus decisions, while preserving stake proportionality and ensuring that every node remains eligible to propose.

Each node $n$ maintains a continuous score $score_n \in [-1, 1]$ representing the recent observed performance of a proposer. Scores are continuously updated according to the classifications produced by the mechanism described in the previous section. We define $decision_n \in \{-1, 0, 1\}$, where: $slow = -1$, $\perp = 0$, and $fast = 1$.

Scores evolve according to the following exponential averaging recurrence:

$$score'_n = trunc_{[-1,1]}(\alpha \cdot decision_n + (1 - \alpha) \cdot score_n)$$

where parameter $\alpha$, $0 \leq \alpha \leq 1$, controls how strongly new observations influence the score. Larger $\alpha$ values make the score react faster to performance changes, whereas smaller $\alpha$ values make scores less sensitive to transient fluctuations. Function $trunc_{[-1,1]}()$ ensures that scores are within interval $[-1, 1]$.

The resulting score is then combined with the node's stake to bias proposer selection:

$$weight_n = stake_n \cdot (1 + C \cdot score_n)$$

where parameter $C$, $0 \leq C < 1$, controls how strongly observed performance influences proposer selection relative to stake. When $C = 0$, proposer selection depends only on the stake. As $C$ increases, higher-performing nodes become proportionally more likely to be selected as proposers. Since $score_n \in [-1, 1]$ and $C < 1$, every node always retains a strictly positive weight and therefore remains eligible to become a proposer.

The deterministic `nextProposer` function of the underlying consensus algorithm is then executed using $weight_n$ instead of the original $stake_n$. As a result, proposer selection continuously adapts to observed consensus performance while preserving the original proposer rotation mechanism.

Finally, since the system is decentralized, parameters $\alpha$ and $C$ must remain consistent across nodes. In systems that support dynamic reconfiguration, these parameters may themselves be updated via consensus and recorded in the replicated state.

### 5.4 Properties and correctness

We now argue that the proposed mechanism satisfies the following properties.

We define a *good run* as an execution in which every honest proposer receives classification votes from all honest processes.

- P.1 The safety and liveness properties of the underlying consensus algorithm are preserved.
- P.2 Proposers associated with faster (slower) consensus decisions become proposers more (less) frequently.

P.3 Changes in the relative performance of proposers over time lead to corresponding adaptations in proposer frequency.
P.4 Every honest node eventually becomes a proposer.
P.5 If all honest processes classify proposer $P$ as $x \in \{\mathit{fast}, \mathit{slow}\}$, and the proposer responsible for choosing the classification is honest and executes in a good run, then $P$ is eventually adopted as $x$.
P.6 A proposer is adopted as $x \in \{\mathit{fast}, \mathit{slow}\}$ only if at least a majority of honest processes classify it as $x$.

*Argument for P.1.* The proposed mechanism does not modify the underlying consensus algorithm. Instead, it piggybacks signed classification votes onto existing protocol messages and processes them independently from consensus execution. Moreover, the mechanism does not introduce any additional coordination that could block consensus progress. Therefore, the safety and liveness properties of the underlying consensus protocol are preserved.

*Argument for P.2.* The score $\mathit{score}_P$ of proposer $P$ increases when the proposer is classified as *fast*, and decreases when it is classified as *slow* (Section 5.3). Since proposer weights are computed as $\mathit{weight}_P = \mathit{stake}_P \cdot (1 + C \cdot \mathit{score}_P)$, higher-performing proposers eventually receive larger weights and therefore become more likely to be selected by the deterministic *nextProposer* function.

*Argument for P.3.* The mechanism continuously updates proposer scores using recent consensus observations. Parameter $\alpha$ controls how strongly new classifications affect the score, while older observations are progressively discounted. Consequently, changes in proposer performance over time are reflected in the corresponding weights and proposer frequencies.

*Argument for P.4.* Every proposer $P$ retains a strictly positive weight since $\mathit{score}_P \in [-1, 1]$ and $C < 1$. Therefore, no proposer is permanently excluded from proposer selection. Since the underlying *nextProposer* mechanism rotates among all positive weights, every honest node eventually becomes a proposer.

*Argument for P.5.* Let $Q$ be the honest proposer responsible for choosing the classification of proposer $P$. Without loss of generality, assume that all honest processes classify $P$ as *fast*. Since there are $2f+1$ honest processes and the run is good, $Q$ receives $2f+1$ matching signed classifications for *fast*, independently of any Byzantine votes. Therefore, the condition for choosing a non-$\perp$ classification is satisfied, and $Q$ includes the classification *fast* together with the supporting signed votes in the proposed value. Since the proposal is decided by consensus, all honest processes eventually adopt the classification *fast*. The argument for *slow* is symmetric.

*Argument for P.6.* Assume proposer $P$ is adopted as *fast*. Then, some proposer $Q$ must have proposed a valid classification supported by at least $\lceil (n + f)/2 \rceil$ matching signed classifications for *fast* (Section 5.2).

Let $k_h$ and $k_b$ denote the number of received *fast* classifications originating from honest and Byzantine processes, respectively. Thus,

$$k_h + k_b \geq \left\lceil \frac{n+f}{2} \right\rceil.$$

Since at most $f$ processes are Byzantine, we have $k_b \leq f$. Therefore,

$$k_h \geq \left\lceil \frac{n+f}{2} \right\rceil - f.$$

Using $n = 3f + 1$, we obtain

$$k_h \geq \left\lceil \frac{4f+1}{2} \right\rceil - f = (2f+1) - f = f + 1.$$

Since there are $2f + 1$ honest processes, at least $f + 1$ honest classifications constitute a majority of honest processes. Therefore, a proposer can only be adopted as *fast* (resp., *slow*) if a majority of honest processes classified it as *fast* (resp., *slow*).

## 6 Experimental Evaluation

To validate the proposed mechanism, we utilized an oracle-based experimental methodology. We first executed the Malachite protocol using standard round-robin rotation to establish a baseline. From the resulting logs, we simulated the score-calculation algorithm to generate an optimized proposer sequence. Finally, we re-executed the protocol using this new sequence to measure performance gains. This iterative process was repeated up to six times (*alg-1* through *alg-6*) to observe score stabilization.

The scoring algorithm used consistent parameters across all tests: uniform node stake, a score range $M = 5$, a 50% consensus threshold ($T = 0.5$), $\alpha = 3$, $\beta = 0.9$, and a 100% contribution factor ($C = 1$). These values were chosen to prioritize rapid convergence in stable network conditions.

We repeated the experimentation with 3 different topologies, respectively cases A, B and C.

**Case A.** In the 13-node AWS topology, we evaluated the iterative convergence of the algorithm (*alg*) against the default rotative system (*def*). As shown in **Fig. 2**, which tracks block times across sequential runs, the high $\alpha$ value allows the scores to stabilize within four iterations. Once stabilized, the algorithm achieves an 8% performance gain primarily by reducing the frequency of the two most distant nodes in the proposer pool. For comparison, a traditional node-removal strategy (*rem-2*) achieves a slightly higher 10% gain but incurs a critical cost: the two most distant nodes were excluded, reducing the committee size from $N = 13$ to $N = 11$, which lowers the Byzantine fault tolerance from $F = 4$ to $F = 3$. Our approach maintains the full committee while achieving comparable performance.

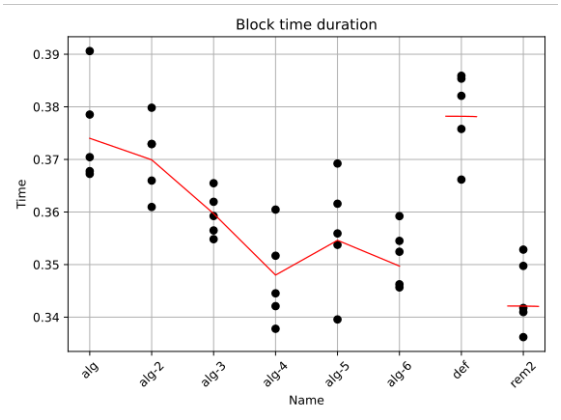



Fig. 2: Experiment with the topology used for measurements Section 3, 13 AWS zones. 5 runs (each one a plot) per configuration. Red line is the average.

**Case B.** To assess robustness under variable network conditions, we utilized datasets from [3] to model two cases, one with 24-hour data and another with 16-month data. In the 14-node 24-hour topology (**Fig. 3**), the scores stabilized in only three steps, yielding a 30% improvement on block time at the 80th percentile and a 15% improvement at the median. This confirms that gains are topology-dependent and also that our mechanism helps filtering out long-tail latencies.

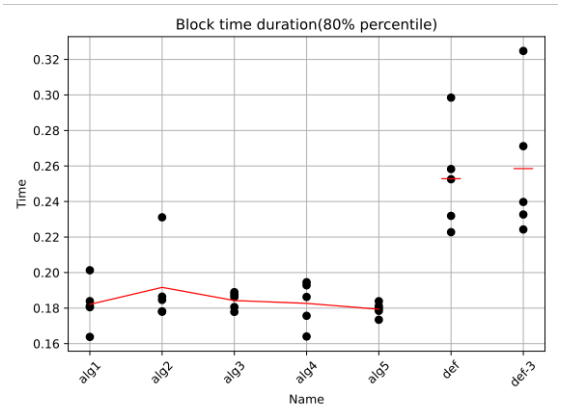



Fig. 3: Experiment with 24-hour measured 14-node topology from [3].

**Case C.** Finally, we evaluated a worst-case scenario using a 21-node topology modeled on 16 months of noisy data (**Fig. 4**). Despite the lack of correlation between latencies of sequential packets, and high latency variability, the algorithm remained robust. It identified subtle patterns to achieve a 6% gain at the 80th percentile and 3% at the median without degrading performance. This stability is crucial for real-world deployment, ensuring that the mechanism provides benefits when patterns exist without introducing overhead during periods of extreme instability.

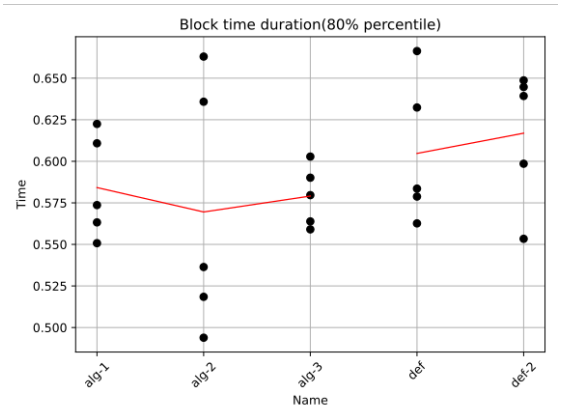



Fig. 4: Experiment with 16 months measures 21 zones topology from [3]. 5 runs per configuration with 10 rotative runs ????? run run?????

## 7 Conclusion

— to be written

Note that this is an experiment with only 13 nodes, while Cosmos, a large scale implementation of Tendermint, is expected to have up to 180 nodes in the committee[10]. It is possible that a single fixed proposer could suffer additional overhead at that scale. Nevertheless, reducing the proposer pool to a smaller subset, excluding only the most disproportionally slow nodes, could still be a realistic approach.

—- In this paper we study how the position of a proposer within a topology affects distributed consensus, and propose a generic mechanism to improve performance through leader selection. As our research focuses on Tendermint, further research on other consensus algorithms is warranted to validate these findings and explore potential edge cases. In particular, algorithms such as HotStuff, with its more involved leader, are expected to have more pronounced leader impact on consensus.

[10] `https://docs.cosmos.network/hub/latest/validators/overview`

Finally, our mechanism is shown to improve performance with no measurable overhead. Nevertheless, several areas could be explored further, such as the impact of individual parameters and possible improvements. In particular, a more nuanced vote could lead to additional performance improvements.

**Disclosure of Interests.** The authors have no competing interests to declare that are relevant to the content of this article.

## References


1. Buchman, E.: Tendermint: Byzantine fault tolerance in the age of blockchains. Master's thesis, University of Guelph (2016)
2. Castro, M., Liskov, B., et al.: Practical byzantine fault tolerance. In: OsDI. vol. 99, pp. 173–186 (1999)
3. Di Perna, V.P., Bernardo, M., Fabris, F., Amaro, S., Matos, M., Schiavoni, V.: Impact of network topologies on blockchain performance. In: Proceedings of the 19th ACM Int. Conf. on Distributed and Event-based Systems. pp. 122–133 (2025)
4. Douceur, J.R.: The sybil attack. In: International workshop on peer-to-peer systems. pp. 251–260. Springer (2002)
5. Du, N., Liang, Z., Huang, Y., Guo, Z., Yang, H., Wang, S.: Performance optimisation method of pbft consensus for supply chain integration svm. In: 2020 7th international conference on dependable systems and their applications (DSA). pp. 371–377. IEEE (2020)
6. Eischer, M., Distler, T.: Latency-aware leader selection for geo-replicated byzantine fault-tolerant systems. In: 2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W). pp. 140–145. IEEE (2018)
7. Huang, D., Huang, Y., Yang, Y.: Improved pbft consensus algorithm based on node evaluation and dynamic management. In: Proceedings of the 2023 4th International Conference on Big Data Economy and Information Management. pp. 851–855 (2023)
8. Kruskal, W.H., Wallis, W.A.: Use of ranks in one-criterion variance analysis. Journal of the American Statistical Association **47**(260), 583–621 (1952). https://doi.org/10.1080/01621459.1952.10483441, https://doi.org/10.1080/01621459.1952.10483441
9. Nakamoto, S.: Bitcoin whitepaper. URL: https://bitcoin. org/bitcoin. pdf-(: 17.07. 2019) **9**, 15 (2008)
10. Qiao, P.: Enhancing manufacturing-as-a-service supply chain transparency with master-slave multi-chain and sqs-pbft. IEEE Access **12**, 186605–186616 (2024)
11. Shan, B., Gao, T., Song, L., Cui, Y.: Grouped byzantine fault-tolerant consensus mechanism based on node behaviour analysis. In: 2025 4th International Symposium on Computer Applications and Information Technology (ISCAIT). pp. 1890–1895. IEEE (2025)
12. Tikhomirov, S.: Ethereum: state of knowledge and research perspectives. In: International symposium on foundations and practice of security. pp. 206–221. Springer (2017)
13. Wahrstätter, A., Ernstberger, J., Yaish, A., Zhou, L., Qin, K., Tsuchiya, T., Steinhorst, S., Svetinovic, D., Christin, N., Barczentewicz, M., et al.: Blockchain censorship. In: Proceedings of the ACM Web Conference 2024. pp. 1632–1643 (2024)

14. Wen, X., Yang, X.: Mapbft: multilevel adaptive pbft algorithm based on discourse and reputation models. The Computer Journal **68**(6), 635–648 (2025)
15. Yin, M., Malkhi, D., Reiter, M.K., Gueta, G.G., Abraham, I.: Hotstuff: Bft consensus with linearity and responsiveness. In: Proceedings of the 2019 ACM symposium on principles of distributed computing. pp. 347–356 (2019)
16. Yu, L., Wu, Y., Lu, J., Li, T.: An adaptive reputation update mechanism for primary nodes in pbft. In: 2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom). pp. 1948–1953. IEEE (2024)
17. Zhang, G., Pan, F., Tijanic, S., Jacobsen, H.A.: Prestigebft: Revolutionizing view changes in bft consensus algorithms with reputation mechanisms. In: 2024 IEEE 40th International Conference on Data Engineering (ICDE). pp. 1930–1943. IEEE (2024)
18. Zhao, F., Wang, Y.: Pbft consensus algorithm based on reward and punishment mechanism. In: 2022 3rd International Conference on Information Science and Education (ICISE-IE). pp. 168–173. IEEE (2022)
19. Zhu, X., Hu, X., Zhu, W.: Rgpbft: A reputation-based pbft algorithm with node grouping strategy. Arabian Journal for Science and Engineering **50**(15), 11837–11850 (2025)